

AI Audiobook Generator

Project Report

PDF / DOCX / TXT to Speech Conversion with AI Text Processing

1. Introduction

The AI Audiobook Generator is a modern, web-based application built with Python and Streamlit that converts static document files - PDFs, Word documents (.docx), and plain-text files (.txt) - into high-quality, spoken-word audio (MP3) using state-of-the-art text-to-speech (TTS) technology. The application bridges the gap between written content and accessible audio formats, enabling users to listen to books, reports, articles, or any written material with minimal effort.

In an era where content consumption is shifting toward audio-first experiences, this project provides an intelligent pipeline that not only converts text to speech but also leverages large language models (LLMs) to clean, summarize, and enhance the extracted text before synthesis. The result is a natural-sounding audiobook that accurately represents the source material.

Key highlights of the project include support for multiple regional voices and languages (English and Hindi), an interactive Streamlit UI with dark and light themes, AI-powered text cleaning and summarization via Google Gemini and Groq (Llama 3), and study tools such as automated notes and question generation. The application is designed to be accessible, requiring no coding knowledge from end-users, and can run locally with minimal setup.

The application also serves as an assistive tool for visually impaired individuals and those with reading disabilities such as dyslexia, enabling them to consume written content in audio format without any specialized hardware.

2. Proposed Methodology

The system follows a layered pipeline architecture that transforms raw document input into a polished audio output. Each layer is independently designed, allowing components to be replaced or upgraded without affecting the rest of the system. The methodology can be divided into five core stages:

2.1 Document Ingestion and Text Extraction

The first stage involves accepting a user-uploaded document and reliably extracting its textual content. The extraction logic in `text_extraction.py` handles three distinct formats:

- PDF files: Extraction is attempted first using `pdfplumber` (which preserves layout and handles columns well), with a fallback to `pypdf` for robustness.
- DOCX files: The `python-docx` library is used to iterate over paragraphs and table cells, preserving document structure.
- TXT files: Raw bytes are decoded through a sequence of common encodings (UTF-8, Latin-1, CP1252) to handle diverse file origins.

Post-extraction, the raw text undergoes cleaning - removing form-feed characters, soft hyphens, and collapsing excessive blank lines - to produce a clean plain-text representation suitable for downstream processing. The system enforces a strict file size limit of 10 MB (defined as `MAX_FILE_SIZE_BYTES` in `ui_1.py`) to prevent memory overload during extraction. If the uploaded file exceeds this threshold, the user is immediately notified with an error message and extraction is skipped entirely. Only PDF, DOCX, and TXT formats are accepted; any other file type raises a `ValueError` with a descriptive message guiding the user to upload a supported format.

2.2 AI-Powered Text Cleaning

The application uses LLM-based cleaning step via the `llm_service.py` module, which implements a waterfall fallback strategy:

- Primary: Google Gemini (new SDK with Client API)
- Secondary: Google Gemini (legacy `GenerativeModel` SDK)
- Tertiary: Groq with Llama 3.1 8B Instant (using `GROQ_llama` key)
- Fallback: Groq with default API key
- Final fallback: Regex-based basic cleaning

The cleaning prompt instructs the model to fix errors while explicitly preserving artistic formatting such as stanza breaks in poetry and paragraph structure, ensuring that the cleaned text retains its original intent. The cleaning step is only triggered when a valid API key is present in the environment. If no key is available, the pipeline silently skips AI cleaning and proceeds with the regex-based fallback, ensuring the application never blocks the user due to missing credentials. Upon successful cleaning, the UI displays a

brief popup notification indicating which provider performed the cleaning - Gemini, Groq, or Basic — and the method is stored in session state (clean_method) so it can be referenced later in the interface caption.

2.3 AI-Powered Summarization

For long documents, users can opt for a summarized audiobook. The summarization module (gemini_helper.py) sends the extracted text to an LLM with a prompt specifying the target word count. The system enforces a hard constraint: the summary must always be shorter than the original document. If AI summarization is unavailable, an extractive summarization fallback (summarize_text_basic) scores and ranks sentences by term frequency weighted by position, selecting the most informative sentences up to the target length.

2.4 Text-to-Speech Synthesis

The cleaned (or summarized) text is passed to the TTS engine in tts.py, which uses the edge-tts library - Microsoft's neural text-to-speech service. To overcome per-request size limitations, the text is intelligently split into chunks of up to 2,000 characters at natural paragraph and sentence boundaries. Each chunk is synthesized asynchronously and the resulting audio segments are concatenated using pydub into a single seamless MP3 file.

The application supports nine distinct voice options across multiple accents and languages, including US Male/Female, British Male/Female, Indian Male/Female, Hindi Male/Female, and a Child voice, each mapped to a specific edge-tts neural voice endpoint.

The voice selection presented to the user is dynamically filtered based on the document language chosen at upload time. If the user selects Hindi as the document language, only Hindi voices (hi-IN-MadhurNeural and hi-IN-SwaraNeural) are shown in the dropdown. For English documents, all non-Hindi voices are listed. This prevents accidental language-voice mismatches that would produce unintelligible audio output. Audio duration is precisely calculated post-generation using pydub's AudioSegment, with a fallback estimate of 2.5 words per second when pydub is unavailable.

2.5 Study Tool Generation

Beyond audio generation, the methodology includes post-processing study tools. Notes are generated using a lightweight extractive approach (filtering meaningful sentences). Questions are generated using the same LLM waterfall strategy, prompting the model to produce insightful comprehension questions. Both notes and questions can be separately converted to audio and downloaded. This creates a complete self-study package from a single document upload - the original audiobook, a summarized version, bullet-point notes with audio, and comprehension questions with audio — all generated within the same session without requiring any additional input from the user.

3. Implementation

The application is implemented entirely in Python and organized across multiple focused modules, each responsible for a specific concern. Below is a detailed description of the implementation of each component.

3.1 Application Entry Point (app.py)

The main application file configures the Streamlit page and orchestrates the three-stage user workflow: Upload, Generate, and Listen. It initializes session state, applies the selected theme, and renders each UI section in order.

The `render_upload_section()` function handles file upload, triggers text extraction, and invokes AI cleaning if an API key is present. It displays the file name, word count, and a toggle to preview the extracted text. A popup notification confirms which cleaning method was used.

The `render_generate_section()` function presents the audio mode selector (Full or Summarized), a slider for target summary length, a voice selector filtered by the document language, and the generate button. On generation, it triggers summarization (if selected), then calls the TTS engine, and stores audio bytes in session state.

3.2 Text Extraction (text_extraction.py)

This module implements format-specific extractors with a common interface. The `extract_text()` function dispatches to the appropriate extractor based on the file extension. The `_extract_pdf()` function applies `pdfplumber` first with custom x/y tolerances for better column detection, falling back to `pypdf`. The `_clean_text()` helper normalizes whitespace and removes control characters. The `get_word_count()` utility provides a simple word count used throughout the UI.

3.3 LLM Service Layer (llm_service.py)

The LLM service is the central abstraction for all AI calls. The `call_llm()` function implements the four-step waterfall:

- Step 1: Attempts the new Google GenAI SDK (`google.genai.Client`) with the provided model name (default: `gemini-2.0-flash`).
- Step 2: Falls back to the legacy `google.generativeai.GenerativeModel` API.
- Step 3: Tries Groq's Python client with the `llama-3.1-8b-instant` model using the `GROQ_llama` environment variable key.
- Step 4: Retries Groq using the standard `GROQ_API_KEY`.

Each step is wrapped in a try/except block. The function returns a tuple of (`response_text`, `provider_name`) or ("", 'failed') if all providers are unavailable. This design ensures the

application remains functional even without API keys, gracefully degrading to rule-based methods.

3.4 AI Helper Functions (`gemini_helper.py`)

This module wraps `call_llm()` with domain-specific prompts for three tasks:

- `clean_text_with_gemini()`: Sends a cleaning prompt and falls back to regex-based `clean_text_basic()`.
- `summarize_text_with_gemini()`: Sends a summarization prompt with target word count and falls back to `summarize_text_basic()`, an extractive TF-scored summarizer.
- `generate_questions_with_gemini()`: Sends a question generation prompt and returns formatted plain-text questions.

The extractive summarizer (`summarize_text_basic`) scores sentences using normalized term frequency, applies a position-decay weight (favoring earlier sentences), and greedily selects sentences until the target word count is reached.

The `generate_questions_with_gemini()` function includes a post-processing step that replaces bullet symbols (`•` and `*`) with hyphens (`-`) to ensure consistent plain-text formatting regardless of which LLM provider generated the response, since different models format lists differently.

3.5 Text-to-Speech Engine (`tts.py`)

The TTS module exposes a single synchronous function, `text_to_speech()`, which internally manages an async event loop using `asyncio.run()`. The text splitting logic (`_split_text_for_tts()`) preserves paragraph boundaries and splits at sentence boundaries when paragraphs exceed the chunk limit, with a final character-level split as a last resort. The `_split_text_for_tts()` function first normalizes the text by collapsing multiple spaces and excessive blank lines before splitting, ensuring clean chunk boundaries. It attempts to split at paragraph boundaries first, then at sentence boundaries using a regex that matches sentence-ending punctuation followed by whitespace, and finally falls back to hard character-level splitting.

Audio chunks are generated sequentially (to avoid rate limits) using `edge_tts.Communicate`. When `pydub` is available, chunks are decoded as MP3 `AudioSegments` and concatenated cleanly. Without `pydub`, raw MP3 bytes are concatenated directly. A `progress_callback` parameter enables the UI to display a real-time progress bar during generation.

3.6 UI Framework (`ui_1.py` and `ui_2.py`)

`ui_1.py` defines the session state initialization (`init_state()`) and the full CSS theming (`apply_style()`). Both themes (light mode and dark mode) style all Streamlit components including file uploaders, dropdowns, buttons, sliders, and text areas using CSS overrides targeting Streamlit's internal `data-testid` attributes.

ui_2.py handles the avatar and audio player rendering. The `avatar_audio_synced_html()` function generates a self-contained HTML/CSS/JS block that embeds the audio as a base64 data URI and uses JavaScript event listeners (`play`, `pause`, `ended`) on the audio element to toggle a CSS animation class on the avatar element, creating the speaking avatar animation effect. If the generated audio file exceeds 1.3 MB in size, the `avatar_audio_synced_html()` function returns 'None' instead of an HTML string, since embedding large audio files as base64-encoded data URIs inside the HTML would bloat the component and risk crashing the Streamlit iframe renderer. In this case, the application falls back to a two-column layout where the static avatar is rendered on the left via `st.markdown()` and the audio is played using Streamlit's native `st.audio()` player on the right.

3.7 Listen & Download Section — `next_app.py`

The `render_listen_section()` function in `next_app.py` renders the complete post-generation UI. It includes:

- The animated avatar player (or standard player as fallback)
- Download MP3 button
- Generate Notes button (uses extractive sentence filtering)
- Generate Questions button (calls AI helper)
- Statistics popover showing word count and audio duration (calculated via `pydub` or estimated at 2.5 words/second)

Notes and Questions sections each include a text area for review, a download button for the text, a button to generate audio from the content, and a download button for the generated audio.

3.8 Environment Configuration

API keys are stored in a `.env` file loaded at startup via `python-dotenv`. The application reads `GEMINI_API_KEY`, `GROQ_API_KEY`, and `GROQ_llama` environment variables. All API key handling is centralized in `llm_service.py`, and the application operates in a degraded-but-functional mode if keys are absent. The application reads three distinct API keys from the `.env` file: `GEMINI_API_KEY` for Google Gemini access, `GROQ_llama` for a dedicated Llama-keyed Groq account, and `GROQ_API_KEY` as a standard Groq fallback. None of these keys are required for the application to start or function - their absence simply causes the corresponding provider to be skipped in the waterfall chain. This design makes the application usable out of the box without any API configuration, with AI features activating automatically once keys are added, making it straightforward to deploy in both development and production environments.

4. Results

The AI Audiobook Generator was tested across multiple document types, lengths, and configurations to validate its end-to-end pipeline. The following outcomes were observed:

4.1 Text Extraction Accuracy

Text extraction performed reliably across all supported formats. PDF extraction via pdfplumber successfully handled multi-column academic papers and formatted reports, while the pypdf fallback handled simpler scanned documents. DOCX extraction correctly parsed both paragraph text and table cell content. TXT extraction handled various encodings including UTF-8, Latin-1, and Windows CP1252 without data loss.

4.2 AI Cleaning and Summarization Quality

When Gemini (gemini-2.0-flash) was available, text cleaning visibly improved OCR quality — removing hyphenation artifacts, correcting spacing, and normalizing punctuation. Summarization produced coherent, fluent summaries that preserved key facts and names within the target word count. The extractive fallback, while less fluent, reliably produced shorter summaries and respected the target length constraint.

4.3 Audio Generation Performance

Edge-TTS generated natural-sounding audio for all nine voice options. Chunking at 2,000-character boundaries eliminated truncation errors. For a typical 5,000-word document, full audio generation completed in approximately 30-60 seconds depending on network latency. Audio duration was accurately calculated via pydub, with the fallback estimate (2.5 words/second) providing a reasonable approximation.

4.4 UI and User Experience

The Streamlit interface rendered correctly in both dark and light modes. The speaking avatar animation synchronized accurately with audio playback events across tested browsers. The progress bar provided real-time feedback during TTS generation. The statistics popover correctly displayed word count and audio duration. Note and question generation worked end-to-end with both AI and fallback modes.

5. Conclusion

The AI Audiobook Generator successfully delivers an end-to-end document-to-audio conversion pipeline that is both accessible and intelligent. By combining robust multi-format text extraction, LLM-powered text enhancement, high-quality neural text-to-speech, and interactive study tools within a polished Streamlit web interface, the project achieves its goal of making written content audibly accessible.

The layered fallback architecture — from Gemini to Groq to regex-based processing — ensures the application remains functional across different deployment environments and API availability scenarios. The modular codebase (`text_extraction`, `llm_service`, `gemini_helper`, `tts`, `ui_1`, `ui_2`) makes it straightforward to extend or replace individual components, such as swapping the TTS provider or adding new LLM backends.

Future enhancements could include support for additional document formats (EPUB, HTML), integration with more TTS providers for expanded language coverage, real-time streaming audio playback as text is synthesized, persistent user history for previously generated audiobooks, and a cloud deployment option with user authentication. Additionally, fine-tuned summarization models specific to academic or technical content could improve summary quality for specialized documents.

Overall, the project demonstrates how modern AI services and open-source libraries can be composed into a practical, user-facing application that delivers measurable value - transforming static documents into engaging, listenable audio experiences with minimal friction for the end user.